

ỨNG DỤNG CÁC MÔ HÌNH QUY HOẠCH TUYẾN TÍNH CHO CÁC LOẠI BÀI TOÁN KNAPSACK

Báo cáo Tiểu luận môn học

Nguyễn Hoàng Đạt - 24001989

Lê Sỹ Thức - 25000312

Nguyễn Hải Anh - 24001968

Trường Đại học Khoa học Tự nhiên
Đại học Quốc gia Hà Nội

Hà Nội, 2026

Bài toán xếp ba lô (Knapsack Problem)

Chọn một tập hợp đồ vật có khối lượng w_i và giá trị v_i xếp vào ba lô sức chứa W sao cho tổng giá trị lớn nhất.

3 biến thể chính:

- **0/1 Knapsack:** Mỗi đồ vật chỉ được chọn tối đa 1 lần ($x_i \in \{0, 1\}$). → NP-khó (Karp, 1972).
- **Unbounded Knapsack:** Số lượng đồ vật chọn là không giới hạn ($x_i \in \mathbb{Z}^+$).
- **Fractional Knapsack:** Đồ vật có thể phân chia, cho phép chọn một phần ($x_i \in [0, 1]$). → Giải được bằng đơn hình hoặc tham lam trong thời gian đa thức.

Các giải thuật cơ sở làm hệ quy chiếu (Baselines)

1. Giải thuật xấp xỉ:

- GreedyFractional: Sắp xếp theo tỷ số density v_i/w_i và chọn tuần tự. Độ phức tạp $\mathcal{O}(n \log n)$.

2. Giải thuật chính xác tổ hợp:

- Backtracking: Duyệt toàn bộ cây nhị phân, độ phức tạp $\mathcal{O}(2^n)$.
- BranchAndBound: Cắt nhánh bằng cận trên Fractional relaxation rất chặt.

3. Quy hoạch động (DP):

- Áp dụng công thức truy hồi dựa trên trạng thái con.
- Độ phức tạp thời gian và bộ nhớ: $\mathcal{O}(n \cdot W)$ (giả đa thức - pseudo-polynomial).
- DPUnbounded tối ưu hóa bộ nhớ xuống mảng 1 chiều kích thước W .

Quy hoạch tuyến tính (LP Relaxation) & Thuật toán Đơn hình

Continuous Relaxation của 0/1 Knapsack

Nới lỏng điều kiện nguyên thành $0 \leq x_i \leq 1$. Bài toán được đưa về dạng mô hình LP tổng quát.

Primal Simplex vs. Dual Simplex:

- **Primal Simplex:** Duy trì tính khả thi gốc (primal feasibility) và cải thiện hàm mục tiêu qua các bước xoay pivot.
- **Dual Simplex:** Duy trì tính khả thi đối ngẫu (dual feasibility) và khôi phục tính khả thi gốc.

Đặc điểm của Dual Simplex

Rất phù hợp để giải quyết các bài toán relaxation khi chèn thêm ràng buộc mới (như trong Branch-and-Bound hay Gomory Cut). Không cần chạy Phase I từ đầu mà có thể **Warm-start** trực tiếp để tìm nghiệm tối ưu mới sau 1 $\rightarrow$ 5 bước xoay pivot.

Quy hoạch nguyên: Lát cắt Gomory (Gomory Cut)

Cơ chế sinh lát cắt Gomory

Từ dòng tối ưu liên tục có biến cơ sở không nguyên $x_i^* = \bar{b}_i \notin \mathbb{Z}$:

$$x_i + \sum_{j \in J} \bar{a}_{ij} x_j = \bar{b}_i$$

Lát cắt Gomory (Gomory fractional cut) loại bỏ nghiệm không nguyên hiện tại:

$$\sum_{j \in J} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor) x_j \geq \bar{b}_i - \lfloor \bar{b}_i \rfloor$$

Thách thức hội tụ số học (Floating-point precision limits):

- Trong Python thuần (số thực kép 64-bit IEEE 754), sai số làm tròn tích lũy qua các phép xoay pivot và khử Gauss dẫn đến tính toán phần lẻ bị lệch nhẹ.
- Gây ra các lát cắt bị nhiễu số học, dẫn đến hiện tượng ****không hội tụ (cycling/infinite loop)**** và làm tăng tỷ lệ timeout của Gomory Cut lên 86,3%.

Simplex Branch-and-Bound (BnB)

- **Nguyên lý:** Phân nhánh trên biến $x_i^* \notin \mathbb{Z}$ bằng cách chèn trực tiếp ràng buộc nhánh $x_i \leq \lfloor x_i^* \rfloor$ và $x_i \geq \lceil x_i^* \rceil$ vào bảng đơn hình hiện tại.
- **Tối ưu hóa:** Giải lại (Warm-start) bằng Dual Simplex cực nhanh.

Điểm nghẽn tính toán (Gaussian Elimination)

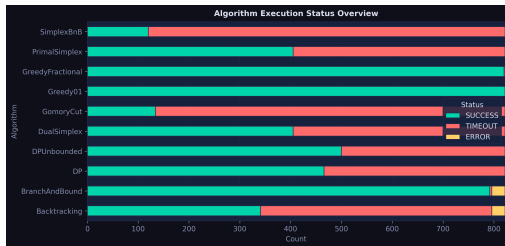
Để tránh ma trận tableau phình to, thuật toán chạy khử Gauss $\mathcal{O}(m^2n)$ để lọc bỏ các ràng buộc dư thừa ở mỗi nút.

- Ở quy mô $n = 750$, với $m \approx 750$ ràng buộc, số phép tính tại mỗi nút đạt xấp xỉ $750^2 \times 750 \approx 4,2 \times 10^8$ phép tính.
- Việc thực thi vòng lặp lồng lớn này bằng Python thuần (không JIT/C compiled) gây ra tỷ lệ timeout cao (85,2%).

Thiết lập thực nghiệm

- **Quy mô:** Tổng cộng 8.220 lượt chạy trên 822 testcase ngẫu nhiên.
- **4 Kịch bản kiểm thử:**
 - **core_exact:** $N \leq 50$, kiểm tra thuật toán chính xác.
 - **core_heuristic:** $N \leq 1000$, so sánh tiệm cận và khả năng chịu tải.
 - **stress:** $W \approx 50.000$, thử thách thuật toán quy hoạch động.
 - **extreme_n_test:** $N \leq 10.000$, kiểm tra giới hạn của giải thuật tham lam.
- **Môi trường:** Python 3.14, NumPy 2.x, SciPy 1.x. Đo lường cách ly tiến trình (Process Isolation) để khử nhiễu từ bộ thu gom rác (Garbage Collector).

Kết quả thực nghiệm: Phân phối trạng thái chạy

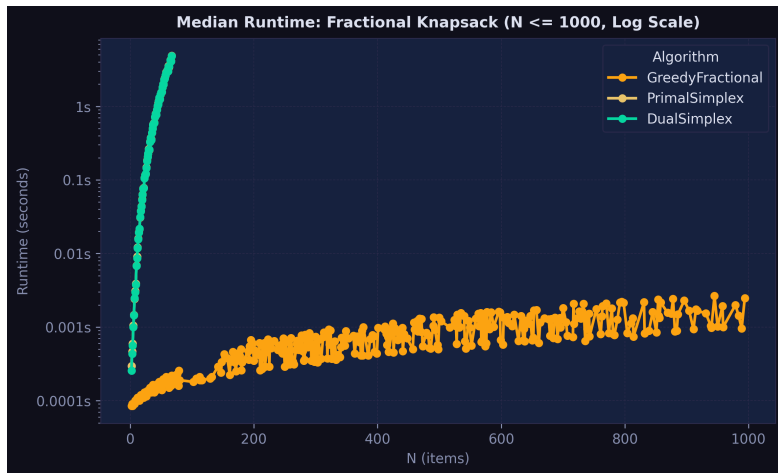


Hình: Trạng thái chạy của 10 thuật toán.

Nhận xét từ Bảng số liệu SUCCESS:

- GreedyFractional và Greedy01: Tốc độ cực nhanh (trung vị $< 0,2$ ms), tỷ lệ thành công 100%.
- BranchAndBound combinatorial: Hiệu năng xuất sắc (trung vị 1,65 ms, timeout 0,1%).
- DualSimplex và PrimalSimplex: Trung vị khoảng 189 ms.
- SimplexBnB và GomoryCut: Thời gian chạy trung bình lớn (1,28 s và 0,65 s), timeout $> 85\%$.

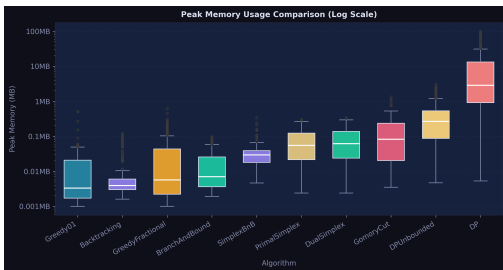
So sánh thời gian thực thi theo kích thước N



Hình: So sánh thời gian chạy trong bài toán Fractional Knapsack (Greedy vs Simplex).

- Sắp xếp tham lam GreedyFractional vượt trội hơn hẳn so với việc giải hệ phương trình đơn hình của Primal và Dual Simplex.

So sánh bộ nhớ sử dụng thực tế

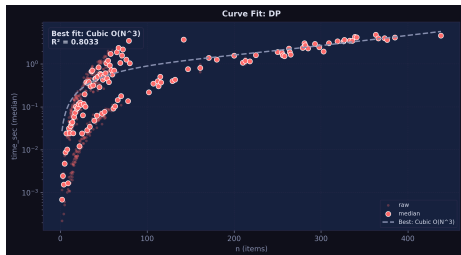


Hình: Phân phối bộ nhớ sử dụng của các thuật toán.

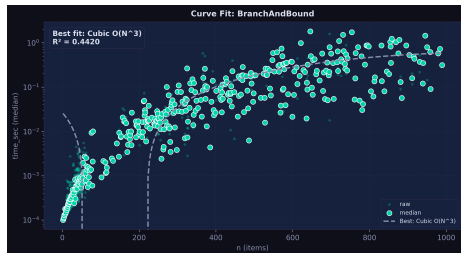
Ưu thế bộ nhớ của Simplex/ILP:

- **DP truyền thống:** Yêu cầu bảng quy hoạch động kích thước $(n + 1) \times (W + 1)$, bộ nhớ bùng nổ theo cấp số nhân $\mathcal{O}(nW)$ lên tới ****88,8 MB****.
- **Simplex/ILP:** Chỉ sử dụng bảng Đơn hình (Tableau) kích thước $(m + 1) \times (n + m + 2)$ phần tử số thực. Bộ nhớ đỉnh luôn được khống chế dưới ****1 MB****, hoàn toàn độc lập với sức chứa túi W .

Khớp đường cong thực nghiệm (Curve Fitting)



Hình: DP: Khớp Bậc ba, $R^2 = 0,79$.



Hình: BranchAndBound: Khớp Bậc ba, $R^2 = 0,50$.

Khớp thực nghiệm số bước xoay pivot của Simplex

Trường hợp xấu nhất của Simplex là hàm mũ (khối Klee-Minty), nhưng thực tế trên bài toán Knapsack chỉ ra số bước xoay pivot tăng trưởng tuyến tính hoặc đa thức bậc cực thấp ($R^2 \geq 0,90$). Điều này chứng minh hiệu năng trung bình cực kỳ tốt của Đơn hình.

Phân tích độ nhạy (Sensitivity Analysis)

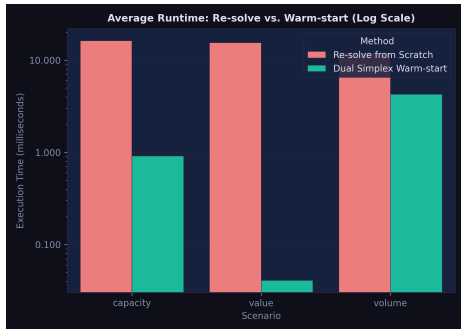
Mục tiêu

Đánh giá hiệu quả của cơ chế **Warm-start** (tận dụng cơ sở tối ưu cũ) bằng Đơn hình ngẫu nhiên khi các tham số của bài toán Knapsack bị biến động, thay vì giải lại từ đầu (Re-solve).

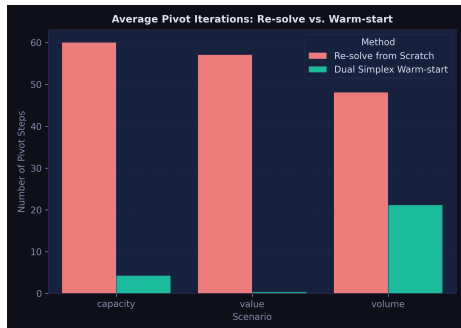
3 Kịch bản biến động tham số thực nghiệm (200 mẫu):

- 1 **Thay đổi sức chứa ba lô W (Capacity RHS):** Tăng W thêm 15%. Tọa độ nghiệm thay đổi $x_B = B^{-1}(b + \Delta b)$. Giữ nguyên tính ngẫu nhiên, áp dụng ****Dual Simplex**** để sửa tính khả thi gốc.
- 2 **Thay đổi hệ số hàm mục tiêu c (Values):** Tăng giá trị c_0 thêm 25%. Giữ nguyên tính khả thi gốc, áp dụng ****Primal Simplex**** nếu mất tính tối ưu ngẫu nhiên.
- 3 **Thêm ràng buộc phụ mới (Volume Constraint):** Thêm ràng buộc $\sum v_i x_i \leq V_{max}$. Nghiệm cũ có thể vi phạm ràng buộc (biến bù âm), giải quyết nhanh bằng ****Dual Simplex****.

Hiệu năng Warm-start bằng Đơn hình đối ngẫu



Hình: Thời gian chạy trung bình (ms, log scale).



Hình: Số bước xoay pivot trung bình.

Hiệu quả gia tốc vượt trội:

- **Kịch bản Value:** Tăng tốc ****770,0 lần**** (Warm-start chỉ mất 0,37 bước xoay pivot so với 57,1 bước khi giải lại từ đầu).
- **Kịch bản Capacity:** Tăng tốc ****24,2 lần**** (xoay pivot 4,3 bước so với 60,1 bước).
- **Kịch bản Volume:** Tăng tốc ****5,6 lần**** (xoay pivot 21,2 bước so với 48,1 bước).

- **Trade-off giữa tốc độ và tính tổng quát:**

- BranchAndBound combinatorial cực kỳ nhanh trên Knapsack cơ bản nhờ tận dụng cấu trúc một ràng buộc duy nhất.
- Simplex/ILP tổng quát bị chậm do chi phí ma trận và khử Gauss, nhưng cực kỳ mạnh mẽ khi giải bài toán có ràng buộc phức tạp hơn (đa chiều, bài toán liên tục) mà các giải thuật tổ hợp chịu bó tay.

- **Cấu trúc cơ sở (basis) làm bộ nhớ hình học:** Cơ sở tối ưu lưu giữ thông tin cấu trúc ma trận của bài toán. Việc bảo toàn cơ sở cũ giúp warm-start giải bài toán biến động động cực nhanh, làm nền tảng cho các cây quyết định quy mô lớn.

- **Giới hạn của DP khi capacity lớn:** Quy hoạch động mất ưu thế hoàn toàn so với Simplex khi sức chứa W cực lớn và số biến n vừa phải (đã chứng minh trong kịch bản stress và core_heuristic).

Kết luận & Hướng phát triển

Các phát hiện định lượng chính:

- Khẳng định hiệu quả đột phá của **Warm-start Đơn hình đối ngẫu** trong phân tích độ nhạy (tăng tốc lên tới hơn 770 lần).
- Chứng minh tính chất tối giản bộ nhớ của Simplex tableau (< 1 MB) so với sự bùng nổ của Quy hoạch động (88, 8 MB).
- Làm rõ lỗi hội tụ do sai số dấu phẩy động trong Python của Gomory Cut và bottleneck khử Gauss tại các nút của Simplex BnB.

Hướng phát triển tương lai:

- Triển khai các cấu trúc ma trận đơn hình bằng thư viện Cython/Numba để biên dịch mã máy, tối ưu hóa phép khử Gauss tại các nút.
- Mở rộng bộ benchmark sang các bài toán tối ưu hóa đa chiều có ràng buộc phức tạp, nơi quy hoạch tuyến tính (LP) thể hiện ưu thế vượt trội.

CẢM ƠN THẦY CÔ VÀ CÁC BẠN ĐÃ LẮNG NGHE!

Kho lưu trữ mã nguồn dự án:

<https://github.com/thucnc7/KnapsackOptimization.git>